

Implementation of Convolutional Encoder and Viterbi Decoder using VHDL

Yin Sweet Wong, Wen Jian Ong, Jin Hui Chong, Chee Kyun Ng, Nor Kamariah Noordin

Department of Computer and Communication Systems Engineering, Faculty of Engineering

Universiti Putra Malaysia, 43400, Serdang, Selangor, Malaysia

yinsweet@gmail.com, owj_87@hotmail.com, chongjinhui@yahoo.com, {mpnck, nknordin}@eng.upm.edu.my

Abstract— This work focuses on the realization of convolutional encoder and adaptive Viterbi decoder (AVD) with a constraint length, K of 3 and a code rate (k/n) of $1/2$ using field-programmable gate array (FPGA) technology. This paper presents a 4-state, radix-2, hard decision AVD which has the ability to decode adaptively through different traceback length (TL). The performance of the implemented AVD is analyzed by using ISE 9.2 and MATLAB simulations. The AVD is targeted to a Xilinx XCV300PQ240-4 FPGA device for hardware realization. The decoder parameter TL can be reconfigured via the implementation of AVD, in accordance with the changing channel noise characteristics of the threshold signal-to-noise ratio (SNR), which is 6 dB. The synthesis results show that the reconfiguration parameter TL of 4 and 15 of AVD implementation has significant difference (>20% improvement) in FPGA device utilization. The results also show that the use of reconfiguration leads to a 28% area occupancy of slice usage improvement over a TL of 15 model compared to a TL of 4 model with tolerable loss of decode accuracy, in accordance with the bit error rate (BER) for real-time voice and video.

Keywords—Convolutional encoder; Viterbi decoder; VHDL; FPGA.

I. INTRODUCTION

In today's digital communications, the reliability and efficiency of data transmission is the most concerning issue for communication channels. Error correction technique plays a very important role in communication systems. The error correction technique improves the capacity by adding redundant information for the source data transmission.

In the late 1940's the approach to error correction coding taken by modern digital communications system started with the ground breaking work of Shannon, Hamming and Golay [1] - [3]. The theoretical limits of reliable communication were defined by the Shannon while Hamming and Golay were developing the first practical error control schemes. Hamming and Golay codes are categorized as linear block codes. In 1954, a new class of linear block codes named Reed-Muller codes were discovered by Muller. Reed-Muller Codes allowed more flexibility in the size of the code and the number of correctable errors per code word [4].

The following discovery code was the cyclic codes. Cyclic codes are also called cyclic redundancy check (CRC) codes primarily used today for the error detection applications rather than for error correction [5]. Bose-Chaudhuri-Hocquenghem (BCH) codes are the important subclass of the cyclic codes

which discovered by Hocquenghem in 1959 and by the team of Bose and Ray-Chaudhuri in 1960. Then the BCH codes were extended to the non-binary case ($q > 2$) by Reed and Solomon in 1960. The Reed-Solomon (RS) codes have found the extensive applications in such systems as Compact Disk (CD) players, Digital Versatile Disk (DVD) players and the Cellular Digital Packet Data (CDPD) standard when the efficient decoding algorithm was introduced by Berlekamp in 1967 [6].

All communication channels are subject to the additive white gaussian noise (AWGN) around the environment. Forward error correction (FEC) techniques are used in the transmitter to encode the data stream and receiver to detect and correct bits in errors, hence minimize the bit error rate (BER) to improve the performance. RS decoding algorithm complexity is relatively low and can be implemented in hardware at very high data rates. It seems to be an ideal code attributes for any application. However, RS codes perform very poorly in AWGN channel.

Due to weaknesses of using the block codes for error correction in useful channels, another approach of coding called convolutional coding had been introduced in 1955 [7]. Convolutional encoding with Viterbi decoding is a powerful FEC technique that is particularly suited to a channel in which the transmitted signal is corrupted mainly by AWGN. It operates on data stream and has memory that uses previous bits to encode. It is simple and has good performance with low implementation cost. The Viterby algorithm (VA) was proposed in 1967 by Andrew Viterbi [8] and is used for decoding a bitstream that has been encoded using FEC code. The convolutional encoder adds redundancy to a continuous stream of input data by using a linear shift register. The Convolutional Encoder and Viterbi Decoder used in the Digital Communications System is shown in Fig. 1.

Most of the Viterbi decoders in the market are a parameterizable intelligent property (IP) core with an efficient algorithm for decoding of one convolutionally-encoded sequence only. In addition, the cost for the convolutional encoder and Viterbi decoder are expensive for a specified design because of the patent issue. Therefore, to realize an adaptive convolutional encoder and Viterbi decoder on a field-programmable gate array (FPGA) board is very demanding. In this paper, we concern with designing and implementing a convolutional encoder and Viterbi decoder which are the essential block in digital communication systems using FPGA technology.

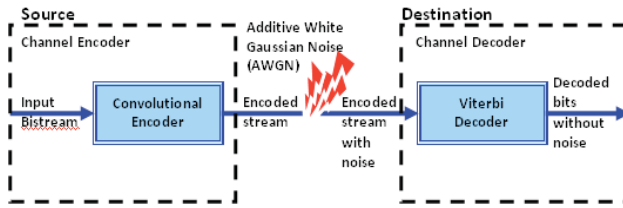


Figure 1. Convolutional encoder and Viterbi decoder.

II. CONVOLUTIONAL ENCODER

A convolutional code is a type of error-correcting code which differs a lot from block codes. First, the former does not have code words made up of distinct data sections and block sections. Instead, redundant bits are distributed throughout the coded data. Second, the encoder of the former contains memory and the n encoder outputs at any given time unit depend not only on the k inputs at that time unit but also on m previous input blocks. Convolutional codes are sometimes referred as trellis codes. Normally, convolutional encoding is simple, but decoding is much more difficult.

Convolutional codes are usually characterized by two parameters and the patterns of n modulo-2 adders. The two important parameters are the code rate and constraint length. The code rate (k/n) where the number of output bits must equal or bigger than the input bits ($n \geq k$), is expressed as a ratio of the number of bits into the Convolutional encoder k to the number of channel symbols output by the Convolutional encoder n in a given encoder cycle. To convolutionally encode data, start with m memory registers, each holding 1 input bit. Unless otherwise specified, all memory registers start with a value of 0. The encoder has n modulo-2 adders, and n generator polynomials, one for each adder. An input bit $m1$ is fed into the leftmost register. Using the generator polynomials and the existing values in the remaining registers, the encoder outputs n bits [1].

As shown in Figure 2, where we have a general encoder designed with a code rate (k/n) of $1/2$ and an information sequence that is being shifted in to the register m of 1 bit at a time. The shift register has a constraint length (K) of 3, equal to the number of stages in the register. The output from the encoder is called code symbols. At initialization all stages in the encoder shall be initially set to zero. The output of the encoder is determined by the generator polynomial equations. Since the complexity of the encoder increases exponentially with the constraint length, none of the encoders uses more than a constraint length of 9, for practical reasons. [9] - [13].

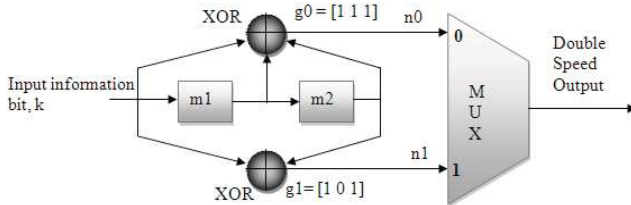


Figure 2. Convolutional encoder with rate (k/n) of $1/2$ and constraint length (K) of 3.

III. VITERBI DECODER

A Viterbi decoder uses the VA for decoding a bitstream that has been encoded using FEC based on a convolutional code. The Viterbi Decoder is used in many FEC applications and in systems where data are transmitted and subject to errors before reception. The VA is commonly used in a wide range of communications and data storage applications. It is used for decoding convolutional codes, in base band detection for wireless systems, and also for detection of recorded data in magnetic disk drives. The requirements for the Viterbi decoder or Viterbi detector, which is a processor that implements the VA, depend on the applications where they are used. The block diagram of Viterbi decoder is shown in Fig. 3. The block diagram consists of the following modules: Branch Metrics, Add-Compare-Select (ACS), register exchange, maximum path metric selection, and output register selection [9], [13].

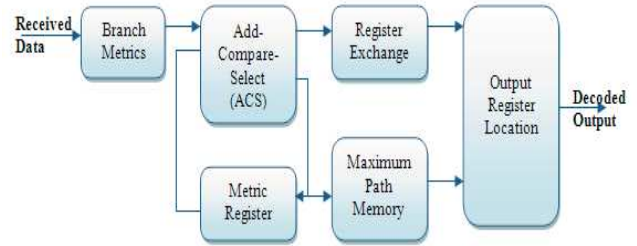


Figure 3. Viterbi decoder block diagram.

IV. ADAPTIVE VITERBI DECODER DECODING ALGORITHMS

VHDL is commonly used as a design-entry language for FPGA and application-specific integrated circuits in electronic design automation of digital circuits. VHDL comes from VHSIC hardware description language, where VHSIC stands for very-high-speed integrated circuit. The process starts with Register Transfer Level (RTL) coding which comprises of Behavioral description and Hardware Description Language (HDL). The convolutional encoder RTL design block diagram is illustrated as Fig. 4. There are three main blocks in the convolutional encoder design. The CONVOLUTIONAL (2, 1, 3) block encodes the IN_BIT (one binary bit) at each clock when the START is HIGH. The outputs of the block are C0 and C1 (both are one binary bit). The 2-BIT MULTIPLEXER block is functioning as a 2-bit parallel-to-serial converter. The encoded codeword is the serial binary data stream and then shows with the LED available on the FPGA board through the DISPLAY_LED block.

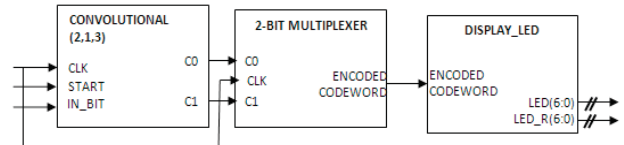


Figure 4. Convolutional encoder RTL design.

The process of adaptive Viterbi decoder (AVD) starts with RTL VHDL coding which comprises of Behavioral description and HDL. The CONTROL block which combines all the subcomponents of AVD RTL design block diagram is illustrated as Figure 9. There are five main blocks in the AVD design which are BMU, ACSU, PMU, SPU & TBU, DU and COUNTER. The CONTROL block is having four input ports including the CLK, START, RESET and MODE of 1 bit standard logic(STD_LOGIC). The CLK is the signal for the synchronization of the circuits. START is a button to enable the decoding process. RESET button reassign all the registers value in the design to initial values. MODE indicates the adaptive traceback length (TL) for decoding process. There is one two bits standard logic vector (STD_LOGIC_VECTOR) input port for the PARALLEL_2BIT_C. It is the parallel 2 bits received codeword from the decoder. There is two 7 bits STD_LOGIC_VECTOR for two 7-segment-LED displays on the FPGA board. There is also 17 bits STD_LOGIC_VECTOR which connected to the expansion connector pins for external LED displays. Besides that, four one bit STD_LOGIC output port are included in the CONTROL architecture. B1 and B0 are the received codeword which has the same value as PARALLEL_2BIT_C. B1 and B0 are connected to the expansion connection pins for external 7-segment LED displays. CLK_OUT and MO are assigned to the Bargraph LED. CLK_OUT is the clocking output while MO is the indicator for TL. The logic '0' indicates it is a TL of 4 decoding mode, on the other hand, logic '1' for TL of 15.

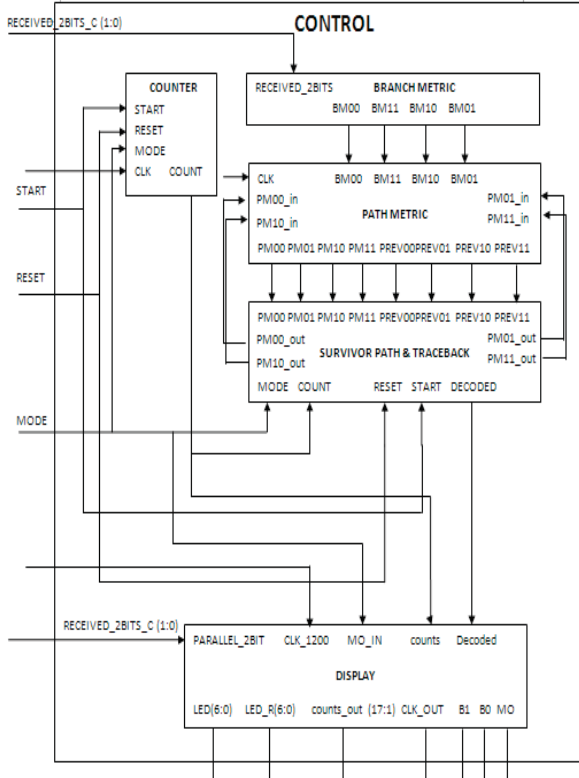


Figure 5. Adaptive Viterbi decoder RTL design.

V. RESULTS AND DISCUSSIONS

A convolutional encoder and Viterbi decoder dynamically-reconfigurable adaptive Viterbi decoder has been presented. The encoder and decoder are implemented on the Xilinx XCV300PQ240-4 FPGA device. The important algorithm parameters have been determined through software simulation and an FPGA-based implementation. The software simulation by using Matlab Software and hardware simulations by using ISE 9.2 software were compared.

In each hardware system design, there is a tradeoff between area and performance. In other words if the resource utilization and used area are increased, the speed and performance are increased relatively as well. The proposed architecture is focused on the area parameter and decreases it as small as possible. The module of this architecture is described with VHDL language and synthesized on Virtex series of FPGA devices. All the synthesis and placement and routing are done with ISE foundation from Xilinx and the whole simulation processes are carried out on Modelsim software. Table 1 presents the resource utilization of the Behavioral Model Viterbi Decoder proposed architecture that includes number of Look-Up-Tables (LUTs), flip flops, slices and so on. On the other hand, Table 2 presents the resource utilization of the Behavioral and Structural Model Viterbi Decoder proposed architecture.

For Behavioral Model traceback length of 4, the number of used Slices is 686 out of 3072 (22%), number of Slice Flip Flops are 207 out of 6144 (3%), number of 4 input LUTs is 1342 out of 6144 (21%), number of bonded IOBs is 8 out of 166 (4%), number of GCLKs is 2 out of 4 (50%). For Behavioral Model traceback length of 15, the number of used Slices is 866 out of 3072 (28%), number of Slice Flip Flops are 207 out of 6144 (5%), number of 4 input LUTs is 1604 out of 6144 (26%), number of bonded IOBs is 8 out of 166 (4%), number of GCLKs is 2 out of 4 (50%). There is significant difference of device utilization between the two traceback length models. The difference number of Slices usage is 180 (26%), the difference number of Slice Flip Flops is 142 (69%) and the difference number of 4 input LUTs is 262 (20%).

TABLE I. DEVICE UTILIZATION REPORT OF BEHAVIORAL MODEL.

Device Utilization Summary (estimated values)						
Logic Utilization	Used		Available		Utilization	
Traceback Length	4	15	4	15	4	15
Number of Slices	686	866	3072		22%	28%
Number of Slice Flip Flop	207	349	6144		3%	5%
Number of 4 input LUTs	1342	1604	6144		21%	26%
Number of bonded IOBs	8	8	166		4%	4%
Number of CLKs	2	2	4		50%	50%

TABLE II. DEVICE UTILIZATION REPORT OF STRUCTURAL MODEL.

Device Utilization Summary (estimated values)						
Logic Utilization	Used		Available		Utilization	
	4	15	4	15	4	15
Traceback Length						
Number of Slices	239	307	3072		7%	9%
Number of Slice Flip Flop	115	176	6144		1%	2%
Number of 4 input LUTs	432	560	6144		7%	9%
Number of bonded IOBs	23	34	166		13%	20%
Number of CLKs	2	2	4		50%	50%

For Behavioral and Structural Model traceback length of 4, the number of used Slices is 239 out of 3072 (7%), number of Slice Flip Flops are 115 out of 6144 (1%), number of 4 input LUTs is 432 out of 6144 (21%), number of bonded IOBs is 23 out of 166 (13%), number of GCLKs is 2 out of 4 (50%). For Behavioral Model traceback length of 15, the number of used Slices is 307 out of 3072 (9%), number of Slice Flip Flops are 176 out of 6144 (2%), number of 4 input LUTs is 560 out of 6144 (9%), number of bonded IOBs is 34 out of 166 (20%), number of GCLKs is 2 out of 4 (50%). There are significant reductions of device utilization between the two traceback length models. The difference number of Slices usage is 68 (28%), the difference number of Slice Flip Flops is 61 (53%) and the difference number of 4 input LUTs is 128 (30%). Comparison of device utilization also has been made between table 1 and table 2. Area is optimized when the VHDL code is written with Structural and Behavioral Style. There is an optimization of 187% of Slice usage, 80% optimization of Slice Flip Flops usage and 210% optimization of the 4 input LUTs usage for traceback length of 4. While for traceback length of 15, there is an optimization of 182% of Slice usage, 98% of Slice Flip Flops usage and 186% optimization of the 4 input LUTs usage.

Instead of wasting channel resource to transmit all sources to achieve the lowest BER among all component sources, more power can be allocated to carry BER sensitive sources and less power allocated to BER insensitive sources. For real time services such as video and voice communications require bounded delay, but having larger acceptable BER (e.g. $BER=10^{-3}$). While Data Traffic is modeled with non-real time service such as ftp service, email, file transfer and web browsing, requires low BER (e.g. 10^{-6}) but tolerates longer delays [14]. The results obtained by using the Matlab coding with rate (k/n) of 1/2 constraint length (K) of 3, hard decision quantizer and two different trellis path or TL of 4 and 15 in AWGN channel as shown in Fig. 6.

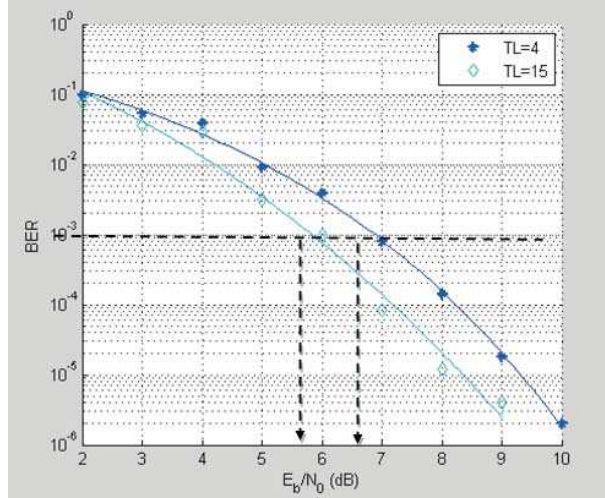


Figure 6. System performance for TLs of 4 and 15.

The threshold of the decoder hence determine at the level of 10^{-3} for the 1 dB power difference optimization for TL of 4 and 15. The decoder will adaptively switch from the TL of 4 mode to TL of 15 mode at the E_b/N_0 of 6 or more than 6 to obtain the BER of 10^{-3} .

VI. CONCLUSIONS

A basic convolutional encoder and an adaptive Viterbi decoder have presented in this paper. The adaptive ability of the Viterbi decoder is according to the threshold level for BER of 10^{-3} . The two different models of Viterbi decoder are according to the TL of the trellis paths. Both models are synthesized and programmed to the FPGA board. The purposed architecture enforces a simple but highly efficient Viterbi arithmetic unit which helps the device utilization or silicon area. Custom implementations in FPGA, allows real-time processing with lower delay (BER of 10^{-3}) and non real-time processing with higher delay (BER of 10^{-6}), providing a good trade-off between performance and area

REFERENCES

- [1] C. E. Shannon, "A mathematical theory of communication," Bell Sys. Tech. J., vol. 27, pp. 379–423 and 623–656, 1948.
- [2] R. W. Hamming, "Error detecting and correcting codes," Bell Sys. Tech. J., vol. 29, pp. 147–160, 1950.
- [3] M. J. E. Golay, "Notes on digital coding," Proc. IEEE, vol. 37, p. 657, 1949.
- [4] D. E. Muller, "Application of boolean algebra to switching circuit design," IEEE Trans. on Computers, vol. 3, pp. 6–12, Sept. 1954.
- [5] E. Prange, "Cyclic error-correcting codes in two symbols," Tech. Rep. TN-57-103, Air Force Cambridge Research Center, Cambridge, MA, Sept. 1957.
- [6] E. R. Berlekamp, R. E. Peile, and S. P. Pope, "The application of error control to communications," IEEE Commun. Magazine, vol. 25, pp. 44–57, Apr. 1987.
- [7] J. Wesdock, and C. Patel, "Formulation of modulation recommendations for future nasa space communications," in IEEE Aerospace Conference, Big Sky, Montana, March 2008.
- [8] A. J. Viterbi, "Error Bounds for Convolutional Codes and an Asymptotically Optimum Decoding Algorithm," IEEE Trans. on Information Theory, Vol. IT-13, pp. 260–269, April 1967.
- [9] W. Chen, "RTL Implementation of Viterbi Decoder," Linköping University, Department of Electrical Engineering, June 2006.
- [10] G. Davis Forney Jr., "The Viterbi algorithm," Proc. IEEE, vol. 61, pp. 268–278, March 1973.
- [11] M. Kivioja, J. Isoaho and L. Vanska, "Design and implementation of Viterbi decoder with FPGAs," Journal of VLSI Signal Processing Systems for Signal, Image, and Video Technology, Kluwer Academic Publishers, vol. 21, no. 1, pp. 5–14, May 1999.
- [12] M. Kling, "Channel Coding Application for CDMA2000 implemented in a FPGA with a Soft processor Core," Linköping University, Department of Electrical Engineering, 2005.
- [13] S. Hema, V. S. Babu and P. Ramesh, "FPGA Implementation of Viterbi Decoder," Proceedings of the 6th WSEAS Int. Conf. on Electronics, Hardware, Wireless and Optical Communications, Corfu Island, Greece, February 16–19, 2007.
- [14] L. Dong, "Cross-layer design for cooperative wireless networks," Drexel Thesis and Dissertations, 4 Dec. 2008.